



Lesson Plan

Time and Space Complexity

Today's Checklist:

1. Time Complexity
2. Different Notions of Complexity
3. Space Complexity
4. Problems on Time & Space complexity

Time Complexity

- Applying the asymptotic analysis to measure the time requirement of an algorithm as a function of input size is known as time complexity. We assume each instruction takes a constant amount of time for time complexity analysis.
- Total time complexity of a program is equal to the summation of all the running time of disconnected fragments.
- **Need for Time Complexity** : When analyzing any algorithm, we need to evaluate the effectiveness of that algorithm. Accordingly, we need to prefer the most optimized algorithm so as to save the time taken by it to execute. An example for the same could be linear search and binary search. Let's suppose we need to search a given value in a sorted array of size 109. This would take 109 iterations for linear search whereas it would just take $\log(109) \sim 30$ iterations for binary search to search the same element, thereby saving a lot of time. Time Complexity for both binary search and linear search will be discussed in further lectures.
- Types of Time Complexity Analysis:
 - a. **Worst Case Time Complexity**: It is that case where the algorithm takes the longest time to complete its execution. It is represented by the Big-O notation. By default we calculate the worst case time complexity when we refer to the time complexity of a code.
 - b. **Best Case Time Complexity**: It is that case where the algorithm takes the least time to complete its execution. It is represented by the Omega Notation (Ω -Notation).
 - c. **Average Case Time Complexity**: As the name suggests, it gives average time for a program to complete its execution. It is represented by the theta notation (Θ -Notation).

Q. Given an array of size $n+1$ consisting of integers from 1 to n . One of the elements is a duplicate in the array. Find that duplicate element.

Input: $a[] = \{1, 3, 2, 3, 4\}$

Output: 3

Explanation: The number 3 is the only repeating element.

//method 1 Use two nested loops. The outer loop traverses through all elements and the inner loop checks if the element picked by the outer loop appears anywhere else.

```
int findRepeating(vector<int> arr)
{
    for (int i = 0; i < arr.size() i++) {
        for (int j = i + 1; j < arr.size(); j++) {
            if (arr[i] == arr[j])
                return arr[i];
        }
    }
    return -1;
}
```

TimeComplexity: $O(n^2)$, traversing the whole array $1+2+3 \dots +n$ time which is of order n^2

SpaceComplexity: $O(1)$, not using any extra space

//method 2 sorting

```
int findRepeating(int arr[], int N) {
    std::sort(arr, arr + N); // sort array

    for (int i = 0; i ≤ N; i++) {
        // compare array element with its index
        if (arr[i] ≠ i + 1) {
            return arr[i];
        }
    }
    return -1;
}
```

Explanation: Sort the given array.

Traverse the array and compare the array elements with its index

if $arr[i] \neq i+1$, it means that $arr[i]$ is repetitive, So Just return $arr[i]$.

Otherwise, the array does not contain duplicates from 1 to $n-1$, In this case, return -1

Time complexity: $O(N * \log N)$

Auxiliary Space: $O(1)$

//method 3 using math

```
int findRepeating(int[] arr, int N)
{
    int sum = 0;
    for (int i = 0; i ≤ N; i++)
        sum += arr[i];
    return sum - (((N + 1) * N) / 2);
}
```

Explanation: Sum of the first n natural number is $n*(n+1)/2$ so we can simply subtract this from the sum of the arr to get the repeating element

Time Complexity: $O(n)$, traversing the whole array to cal the sum

Space Complexity: $O(1)$, not using any extra space

NOTE : Time Complexity can only be calculated on same programming device

Notations for different types of Time Complexity

Asymptotic Notations: Asymptotic Notations are mathematical tools that allow you to analyze an algorithm's running time by identifying its behavior as its input size grows.

There are mainly three asymptotic notations:

Big O Notation (O-notation):

Describes the worst-case scenario of an algorithm's time complexity.

Represents the upper bound on how the algorithm's performance scales with input size.

Example: $O(n^2)$ means the algorithm's time grows no faster than n^2 for input size n .

Big Omega Notation (Ω -notation):

Describes the best-case scenario of an algorithm's time complexity.

Represents the lower bound on how the algorithm's performance scales with input size.

Example: $\Omega(n)$ means the algorithm's time is at least linear for input size n .

Big Theta Notation (Θ -notation):

Describes both the upper and lower bounds of an algorithm's time complexity.

Represents a tight bound on how the algorithm's performance scales with input size.

Example: $\Theta(n)$ means the algorithm's time grows linearly and is neither faster nor slower than a linear function for input size n .

Q. Calculate the time complexity for iterating in a loop.

```
for (int i = 0; i < n; i++) {  
    cout<<"PhysicsWallah\n";  
}
```

Time Complexity: $O(n)$

Explanation: we previously said if the loop is running n times, so time complexity is $O(n)$, here we can verify that we run the following code I love coding will be printed n times which indeed tells us n operations have been performed, so time complexity is $O(n)$

Q. What if this time we increment the pointer by 2?

```
for(int i = 0; i < n; i+=2) {
    cout << "PhysicsWallah\n";
}
```

Time: $O(n/2) \sim O(n)$

Q. Calculate the time complexity for traversing 2 arrays of size n and m.

```
int a[n], b[m];
for(int i = 0; i < n; i++) {
    a[i] = i;
}
for(int i = 0; i < m; i++) {
    b[i] = m-i;
}
```

Time: $O(n+m)$, to traverse $n+m$ elements

Q. What if this time we traverse them in a nested manner?

```
for(int i = 0; i < n; i++) {
    for(int j = 0; j < m; j++) {
        cout<<"okay";
    }
}
```

Time: $O(n*m)$, this time will traverse the m element each n times so $n*m$

Q. Calculate the time complexity for the below code snippet.

```
int c = 0;
for(int i = 1; i ≤ n; i*=k) {
    c++;
}
```

Time: $O(\log n)$

Q. Calculate the time complexity for the below code snippet.

```
int c = 0;
for(int i = 0; i < n; i++) {
    for(int j = i+1; j < m; j++) {
        c++;
    }
}
```

Time: $O(n^2)$, $n-1+n-2+\dots+1=n*(n-1)/2 \sim n^2$

Space Complexity

Q. Calculate the space complexity for the below code snippet.

```
int a[n];
for(int i = 0; i < n; i++) {
    a[i]++;
}
```

Space: $O(n)$, for the array

Q. What will be the space complexity if we just traverse without creating any array?

```
int c = 0;
for(int i = 0; i < n; i++) {
    c++;
}
```

Space: $O(1)$, only const space used

Q. Calculate the space complexity for the below nested loop code snippet.

```
vector<int> a(n);
vector<int> b(m);
for(int i = 0; i < n; i++) {
    for(int j = 0; j < m; j++) {
        a[i]++; b[j]++;
    }
}
```

Space: $O(n+m)$, for space used by vectors

Space Complexity of creating a 2 dimensional matrix

Ans: $O(n*m)$

Q. What will be the space complexity if we create 3 arrays of the same size?

```
int a[n], b[n], c[n];
for(int i = 0; i < n; i++) {
    c++;
}
```

Space: $O(3n) \sim O(n)$

Q. Calculate the time and space complexity for the below nested loop code snippet.

```
int a[n][n/2];
for(int i = 1; i < n; i*=2) {
    for(int j = 0; j < n/2; j++) {
        A[i][j]++;
    }
}
```

Space: $O(n \cdot n/2) \sim O(n^2)$

Q. Calculate the time complexity for the following code snippet.

```
int c = 0;
for(int i = 1; i < n; i += i) {
    c++;
}
```

Space: $O(\log n)$, each time i being doubled

Q. Calculate the time complexity for the following code snippet.

```
int c = 0;
for(int i = 1; i < n; i += i) {
    for(int j = 0; j < i; j++) {
        c++;
    }
}
```

Ans: $O(n)$

Explanation: The value of outer loop will go like 1,2,4,8 ... , 2^k as in prev Ques While the inner loop will run i items. So for each i the number of operation will be Equal to i thus number of operation are $1+2+4+8+ \dots + 2^k$ such $2^k < n$ so it will nearly go upto So a total of $\log n$ terms, now of series $1+2+4+ \dots + 2^k$ is given by $2^{k+1}-1$ and k is of order $\log n$ 2^k is of order n thus $2^{k+1}-1$ is of order n as well so the time complexity is $O(n)$

Q. Calculate the time complexity for the following code snippet.

```
int c = 0;
for(int i = 1; i < n; i += i) {
    for(int j = n; j ≥ 0; j--) {
        c++;
    }
}
```

Ans: $O(n \log n)$

Q. Calculate the time complexity for the following code snippet.

```
int c = 0;
for(int i = 1; i < n; i *= 2) {
    for(int j = 0; j < i; j++) {
        c++;
    }
}
```

Ans: $O(n)$

Explanation: outer loop will go like $i=1, 2, 4, 8 \dots 2^k$ such that 2^k is less than n so k is of order $\log(n)$ and inner loop will run i times for each i that gives $1+2+4+8+16+\dots K$ times $\Rightarrow 2^{\log n} - 1 \Rightarrow n$ operations

Q. Calculate the time complexity for the following code snippet.

```
int c = 0;
for(int i = 1; i * i < n; i *= 2) {
    for(int j = 0; j < i; j++) {
        c++;
    }
}
```

Ans: $O(\sqrt{n})$

Explanation: The inner loop will run i times for each loop and the outer loop will go from $i=1, 2, 4, 8 \dots 2^k$ such that $(2^k)^2 < n$ so $\text{appx } 2^{(2k)} = n$ thus $2k = \log(n), k = \log(n)/2$
So outer loop will run $\log(n)/2$ times thus sum of series is $1+2+4 \dots \log(n)/2$ times
Also $\log(n)/2$ can be written as $\log(\sqrt{n})$, as we know sum of series $1+2+4 \dots k$ times is $2^{k+1} - 1$ thus here it will be of order $2^{\log(n)/2}$ which equals $\sqrt{n}$

Q. Calculate the time complexity for the following code snippet.

```
int c = 0;
for(int i = 1; i * i < n; i += i) {
    for(int j = n; j > i; j--) {
        c++;
    }
}
```

Ans: $O(n \log n)$

Explanation: As explained previously the outer loop will run $\log(\sqrt{n})$ times and the inner loop is running $n-i$ times for each i so time complexity, so its number of operations will be, $n-1 + n-2 + n-4 + \dots \log(\sqrt{n})$ times $\Rightarrow n(\log(\sqrt{n}) - (1+2+4+\dots)) \Rightarrow n(\log(\sqrt{n}) - \sqrt{n}) \Rightarrow n/2\log(n) - \sqrt{n}$, so in big O notation $n/2\log(n)$ term will dominate so time complexity will be $O(n/2\log n)$ or simply $O(n \log n)$

Q. Calculate the time complexity for the following code snippet.

```
int c = 0;
for(int i = 2; i < n; i *= i) {
    c++;
}
```

Ans: $O(\log n)$

Explanation: i is being squared each time so $i \Rightarrow 2, 4, 8, \dots 2^k$, such that $2^k < n$. So $k < \log(n)$ hence there will total $\log(n)$ operation so time complexity is $O(\log n)$

Q. Calculate the time complexity for the following code snippet.

```
int c = 0;
for(int i = 2; i * i < n; i *= i) {
    c++;
}
```

Ans: $O(\log(\sqrt{n}))$ or simply $O(\log n)$

Explanation: $\Rightarrow i$ will go like $2, 4, \dots 2^k$, such that $(2^k)^2 < n$ or simply $2^{(2k)} < n$ so $k < \log(n)/2$, so k is of order $\log(\sqrt{n})$ so time complexity is $\log(\sqrt{n})$ which is nearly of Order $\log(n)$



**THANK
YOU !**

